Understanding Dependency Injection Lifetime in ASP.Net Core

Overview

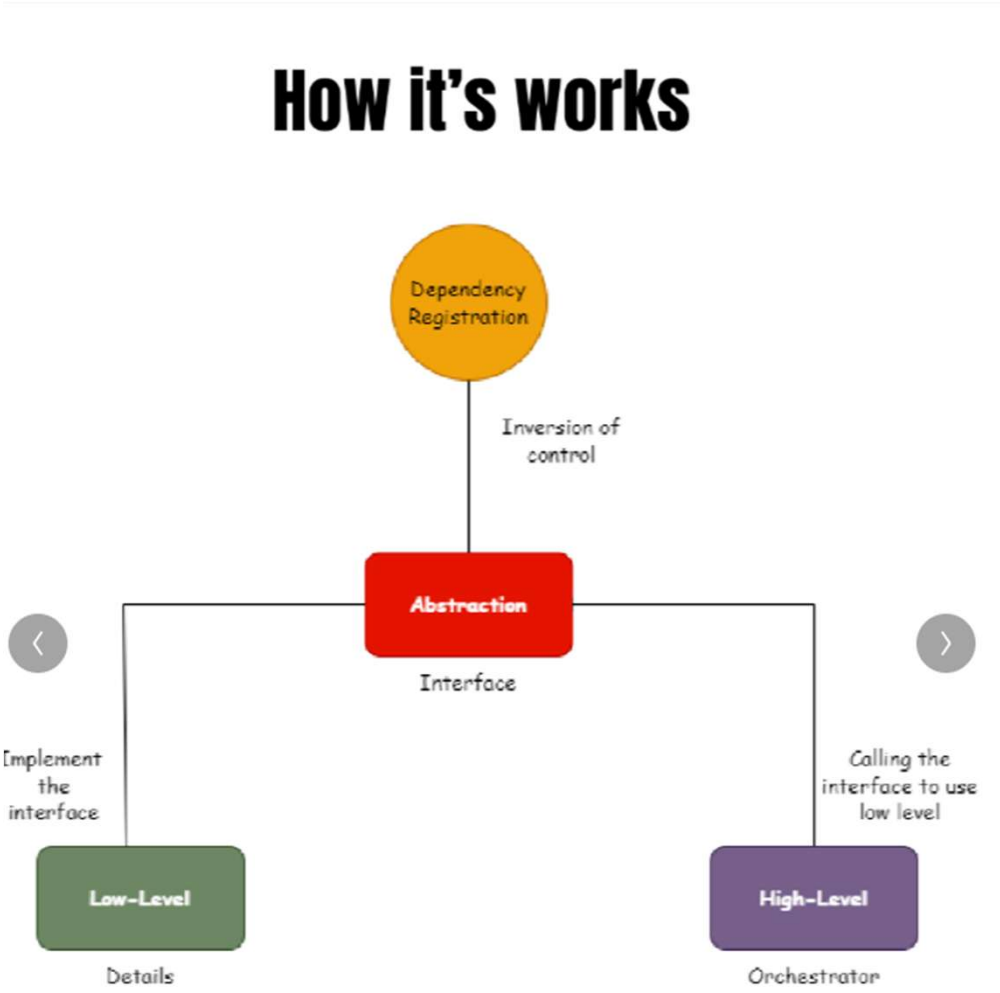


Dependency Injection

Dependency Injection (DI) Life Time

Dependency Injection

Dependency injection (DI) is a core, built-in feature of ASP.NET Core that promotes loosely coupled, testable, and maintainable code. It works by providing a class with its required dependencies from an external source (a built-in Inversion of Control container), rather than having the class create them itself



Dependency Injection (DI) Life Time

Understanding the life cycle of Dependency Injection (DI) is very important in ASP.Net Core applications. As we know, Dependency injection (DI) is a technique for achieving loose coupling between objects and their collaborators, or dependencies.

- Most often, classes will declare their dependencies via their constructor, allowing them to follow the Explicit Dependencies Principle. This approach is known as "constructor injection".
- To implement dependency injection, we need to configure a DI container with classes that is participating in DI.
- DI Container has to decide whether to return a new instance of the service or provide an existing instance. In program class, we perform this activity.
- The lifetime of the service depends on when the dependency is instantiated and how long it lives. And lifetime depends on how we have registered those services.

The below three methods define the lifetime of the services.

- **AddTransient**

Transient lifetime services are created each time they are requested. This lifetime works best for lightweight, stateless services.

- **AddScoped**

Scoped lifetime services are created once per request.

- **AddSingleton**

Singleton lifetime services are created the first time they are requested (or when Program class is run if you specify an instance there) and then every subsequent request will use the same instance.

Methods for Dependency Injection (DI) Life Time in ASP.NET Core Web API

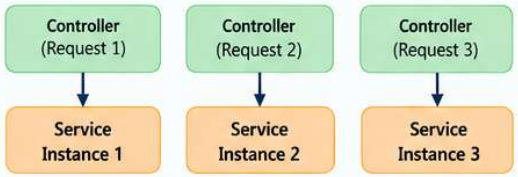
In ASP.NET Core, services are registered in the DI container using different lifetime methods: **AddTransient(), AddScoped(), AddSingleton()**

1. Transient Lifetime

AddTransient<TService, TImplementation>()

- A new instance is created every time the service is requested.
- Each time you inject the service, you get a different instance.
- Best for lightweight, stateless services.

Transient – New instance for every request



Registration (in Program.cs)

```
builder.Services.AddTransient<IProductService, ProductService>();
```

Example Service

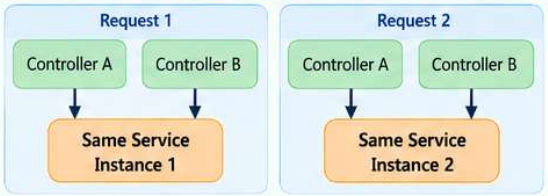
```
public interface IProductService
{
    Guid Id { get; }
}
public class ProductService : IProductService
{
    public Guid Id { get; } = Guid.NewGuid();
}
```

2. Scoped Lifetime

AddScoped<TService, TImplementation>()

- A single instance is created per HTTP request.
- The same instance is shared within the same request pipeline (across controllers, services, etc.).
- A new instance is created for each new request.
- Commonly used for services that work with a database (e.g., DbContext).

Scoped – Same instance within a request



Registration (in Program.cs)

```
builder.Services.AddScoped<IProductService, ProductService>();
```

Example Service

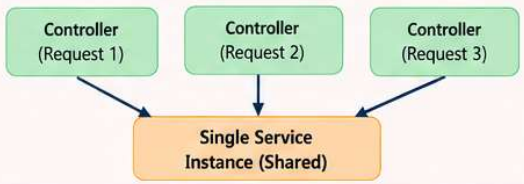
```
public interface IProductService
{
    Guid Id { get; }
}
public class ProductService : IProductService
{
    public Guid Id { get; } = Guid.NewGuid();
}
```

3. Singleton Lifetime

AddSingleton<TService, TImplementation>()

- A single instance is created for the entire application lifetime.
- The same instance is shared across all requests.
- Use only for stateless and thread-safe services.
- Often used for caching, configuration, or shared resources.

Singleton – Single instance for entire application



Registration (in Program.cs)

```
builder.Services.AddSingleton<IProductService, ProductService>();
```

Example Service

```
public interface IProductService
{
    Guid Id { get; }
}
public class ProductService : IProductService
{
    public Guid Id { get; } = Guid.NewGuid();
}
```

Lifetime Method	Instance Creation	Instance Shared With	Typical Use Cases	Code
AddTransient()	New instance every time	Not shared	Lightweight, stateless services	`builder.Services.AddTransient<IService, Service>();`
AddScoped()	One instance per request	Same within a request	DbContext, request-specific services	`builder.Services.AddScoped<IService, Service>();`
AddSingleton()	Single instance for app lifetime	Shared across all requests	Caching, configuration, shared resources	`builder.Services.AddSingleton<IService, Service>();`

Real-World Example of (DI) Life Time

1. AddTransient() – “Use & Throw” (Like Disposable Cups)

Imagine you go to a tea stall:

Every time you ask for tea → you get a new disposable cup
After use → it’s thrown away

👉 No reuse at all.

Use When:

- No state required
- Lightweight objects
- Logging, helper classes

2. AddScoped() – “Per Request” (Like Restaurant Table)

You go to a restaurant:

You are assigned one table
Everything you order → comes to the same table
When you leave → table is cleared

👉 Same instance within one request only

Use When:

- Database operations (VERY IMPORTANT)
- Unit of Work pattern
- Request-based logic

3. AddSingleton() – “One for Entire App” (Like CEO)

A company has one CEO:

Everyone talks to the same person
Exists throughout the company’s lifetime

👉 **Only ONE instance for entire application**

Use When:

- Configuration data
- Caching
- Logging service (global)

AddSingleton(), AddScoped(), and **AddTransient()** are methods used in ASP.NET Core's dependency injection system (IServiceCollection) to register services with different lifetimes. Let's compare them in terms of throughput, scalability, and statefulness:

📊 Side-by-Side Comparison

Aspect	AddTransient	AddScoped	AddSingleton
Scalability	⭐⭐⭐⭐ High	⭐⭐⭐ Medium-High	⭐⭐⭐⭐⭐ Very High
Throughput	⭐⭐⭐ Medium	⭐⭐⭐⭐ High	⭐⭐⭐⭐⭐ Very High
Statefulness	❌ Stateless	⚠️ Semi-Stateful (per request)	⚠️ Highly Stateful (global)
Memory Usage	❌ Higher (many objects)	⚠️ Moderate	✅ Lowest (single instance)
Thread Safety Required	❌ No	❌ No (per request safe)	✅ YES (critical)

References

https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection

https://henriquesd.medium.com/dependency-injection-and-service-lifetimes-in-net-core-ab9189349420

https://www.c-sharpcorner.com/article/service-lifetime-dependency-injection-asp-net-core/

https://endjin.com/blog/2022/09/service-lifetimes-in-aspnet-core

https://code-maze.com/dependency-injection-lifetimes-aspnet-core/

https://levelup.gitconnected.com/dependency-injection-lifetime-7824f366d5ea

https://dotnetcorecentral.com/blog/service-lifetime/